

Actividad 5 - PromeHub

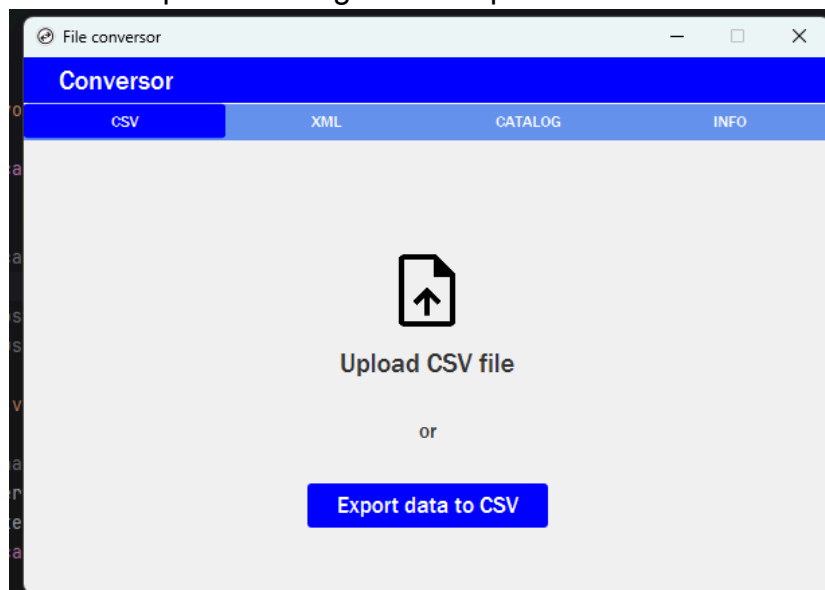
Integrantes: Daniel Riopedre, Sergio Blázquez y Rodrigo García

ANTES DE CARGAR LOS ARCHIVOS

Ventana de conversión CSV

Se pueden realizar dos opciones:

- Subir un CSV
- Convertir los datos que el catálogo tiene importados a CSV

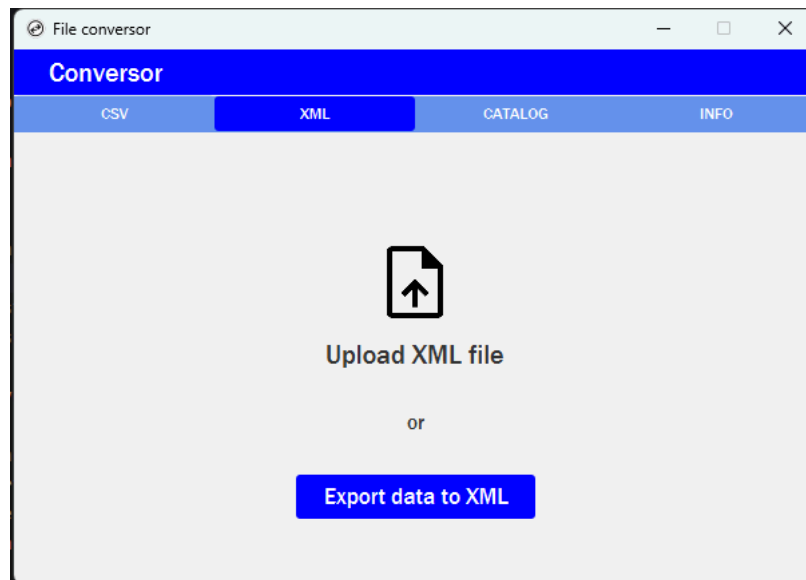


Ventana de conversión XML

Se pueden realizar dos opciones:

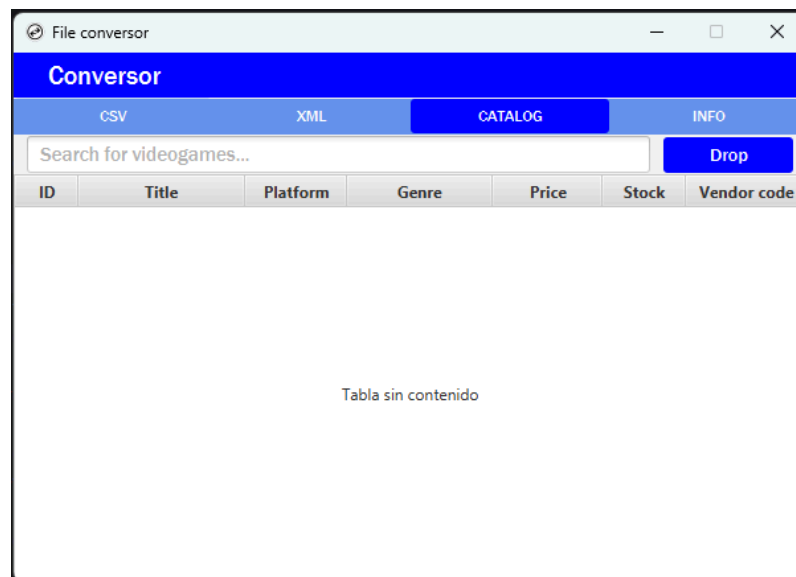
- Subir un XML
- Convertir los datos que el catálogo tiene importados a XML

Nota: a la hora de exportar los datos del catálogo evidentemente el código del proveedor no se exporta debido a que es requerimiento del programa.



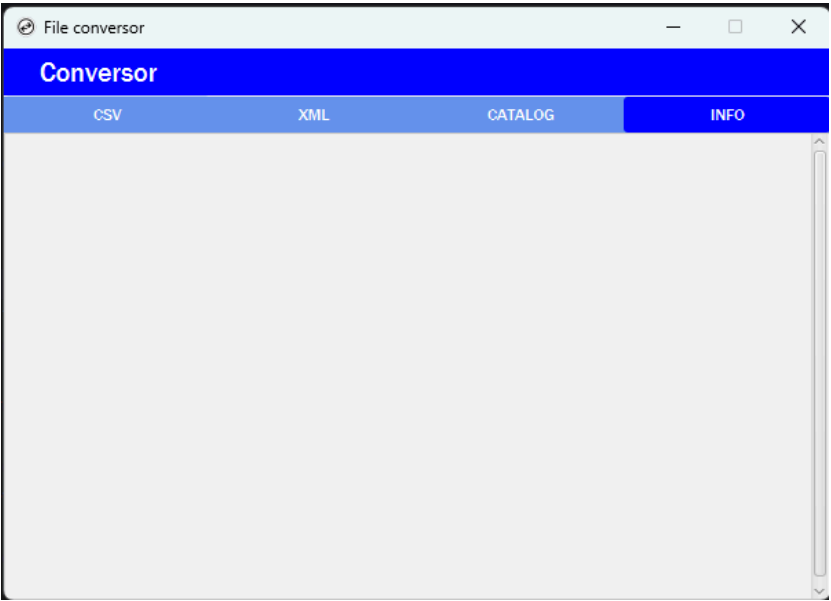
Ventana de conversión catálogo

- Aquí se mostrarán posteriormente los videojuegos cargados mediante el csv o xml.
- Los videojuegos se podrán buscar/filtrar mediante el buscador que es dinámico, es decir que se actualiza cada vez que escribimos o borramos algo.
- El botón de drop sirve para borrar la búsqueda y cuando no hay nada se muestra todo el catálogo



Ventana de información de archivos

- Al principio no tendrá ninguna información pero a medida que el programa trabaja con archivos se irá rellenando de todos aquellos que se utilizaron en el programa (archivos importados tanto exportados)



DESPUÉS DE CARGAR LOS ARCHIVOS

- **CARGA Y CONVERSIÓN DE DATOS**

Carga de CSV

- Solo se pueden cargar CSVs de siete columnas cuyos atributos tengan los mismos tipos que los de la clase Videojuego.

```
id,titulo,plataforma,genero,precio,stock,codProveedor
2,g,g,roberto,67.0,67,67
2,g,g,roberto,67.0,67,null
2,g,g,roberto,67.0,67,null
2,g,g,roberto,67.0,0,null
6,g,g,roberto,0.0,0,null
2,g,g,roberto,67.0,67,67
2,g,g,roberto,67.0,67,67
```

File conversor

—□×

Conversor

CSVXMLCATALOGINFO

Search for videogames...

Drop

ID	Title	Platform	Genre	Price	Stock	Vendor code
2	g	g	roberto	67.0	67	67
2	g	g	roberto	67.0	67	null
2	g	g	roberto	67.0	67	null
2	g	g	roberto	67.0	0	null
6	g	g	roberto	0.0	0	null
2	g	g	roberto	67.0	67	67
2	g	g	roberto	67.0	67	67

Conversión de CSV a XML

- CodProveedor no se mantiene ya que la clase Videojuego tiene el atributo con el mismo nombre marcado con la anotación @XmlTransient.

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<catalogo>
  <videojuego id="2">
    <titulo>g</titulo>
    <plataforma>g</plataforma>
    <genero>roberto</genero>
    <precio>67.0</precio>
    <stock>67</stock>
  </videojuego>
  <videojuego id="2">
    <titulo>g</titulo>
    <plataforma>g</plataforma>
    <genero>roberto</genero>
    <precio>67.0</precio>
    <stock>67</stock>
  </videojuego>
  <videojuego id="2">
    <titulo>g</titulo>
    <plataforma>g</plataforma>
    <genero>roberto</genero>
    <precio>67.0</precio>
    <stock>67</stock>
  </videojuego>
  <videojuego id="2">
    <titulo>g</titulo>
    <plataforma>g</plataforma>
    <genero>roberto</genero>
    <precio>67.0</precio>
    <stock>0</stock>
  </videojuego>
  <videojuego id="6">
    <titulo>g</titulo>
    <plataforma>g</plataforma>
    <genero>roberto</genero>
    <precio>0.0</precio>
    <stock>0</stock>
  </videojuego>
  <videojuego id="2">
    <titulo>g</titulo>
    <plataforma>g</plataforma>
    <genero>roberto</genero>
    <precio>67.0</precio>
    <stock>67</stock>
  </videojuego>
  <videojuego id="2">
    <titulo>g</titulo>
    <plataforma>g</plataforma>
```

Carga de XML

- CodProveedor aparece como vacío debido a que es requerimiento de la aplicación.

File conversor

— □ ×

Convertor

CSVXMLCATALOGINFO

Search for videogames...

Drop

ID	Title	Platform	Genre	Price	Stock	Vendor code
2	g	g	roberto	67.0	67	
2	g	g	roberto	67.0	67	
2	g	g	roberto	67.0	67	
2	g	g	roberto	67.0	0	
6	g	g	roberto	0.0	0	
2	g	g	roberto	67.0	67	
2	g	g	roberto	67.0	67	

Conversión de XML a CSV y carga

- CodProveedor se guarda como null.

```
id,titulo,plataforma,genero,precio,stock,codProveedor
2,g,g,roberto,67.0,67,null
2,g,g,roberto,67.0,67,null
2,g,g,roberto,67.0,67,null
2,g,g,roberto,67.0,0,null
6,g,g,roberto,0.0,0,null
2,g,g,roberto,67.0,67,null
2,g,g,roberto,67.0,67,null
```

2	g	g	roberto	67.0	67	null
2	g	g	roberto	67.0	67	null
2	g	g	roberto	67.0	67	null
2	g	g	roberto	67.0	0	null
6	g	g	roberto	0.0	0	null
2	g	g	roberto	67.0	67	null
2	g	g	roberto	67.0	67	null

Buscar por ID

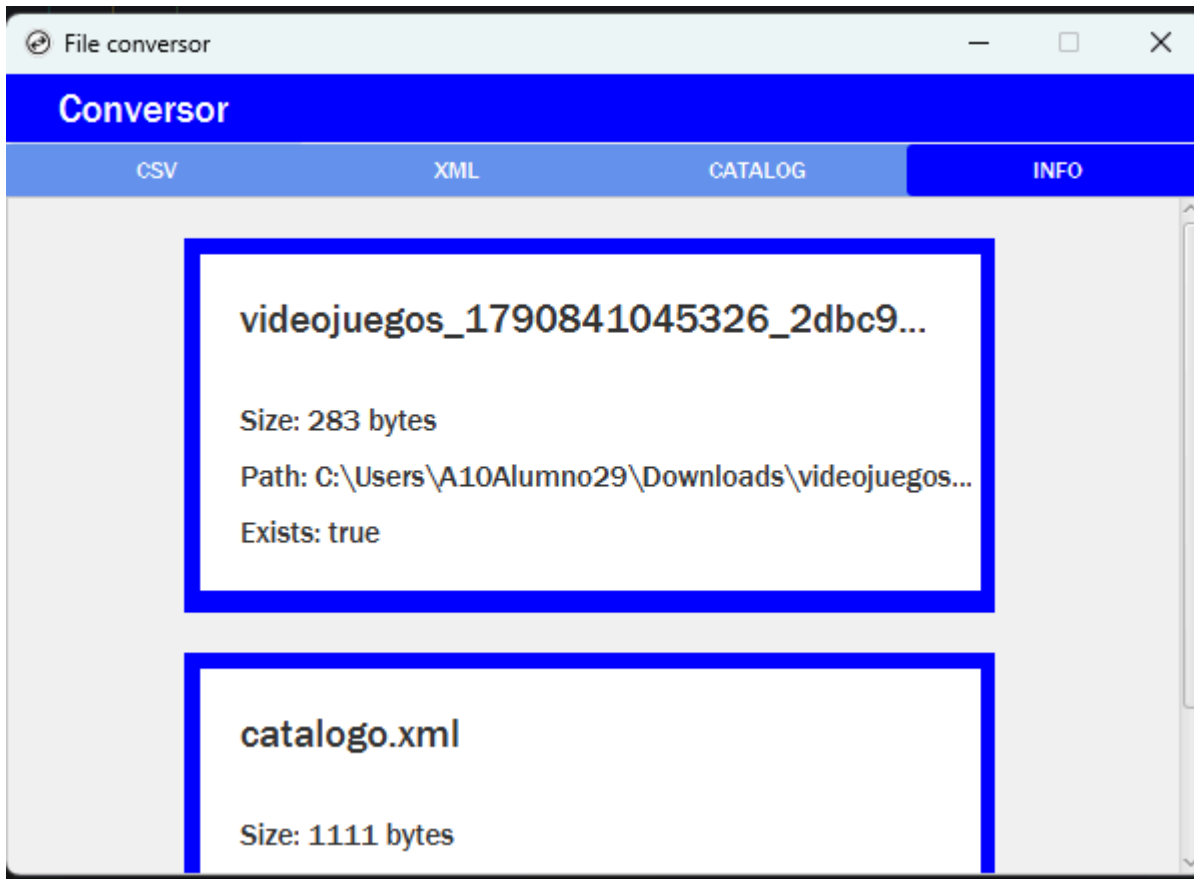
- CodProveedor aparece como vacío (como ya se ha explicado con anterioridad).

[illegible]

Información de los archivos

- Se muestra el tamaño la ruta absoluta y si existe el archivo

Nota: Hemos detectado que si el nombre del archivo es demasiado largo se trunca así que eso podría arreglarse como propuesta de mejora.



CLASES Y MÉTODOS

Videojuego

Atributos y anotaciones

- XmlAccessorType controla la manera en la que se guarda el archivo. AccessType.FIELD causa que todos los atributos no marcados como @XmlTransient se puedan guardar.
- Los atributos marcados como @XmlElement y @XmlAttribute siempre son exportados al serializar la clase a XML.

```
@XmlRootElement(name = "videojuego")
@XmlAccessorType(XmlAccessType.FIELD)
public class Videojuego {

    4 usages
    @XmlAttribute
    private int id;

    4 usages
    @XmlElement
    private String titulo;

    4 usages
    @XmlElement
    private String plataforma;

    4 usages
    @XmlElement
    private String genero;

    4 usages
    @XmlElement
    private double precio;

    4 usages
    @XmlElement
    private int stock;

    4 usages
    @XmlTransient
    private String codProveedor;
```

Catálogo

- Aquí es donde se almacenan las instancias de Videojuego que se generan al cargar datos.

```
package es.promehub.backend;

import ...

12 usages
@XmlRootElement(name = "catalogo")
@XmlAccessorType(XmlAccessType.FIELD)
public class Catalogo {

    3 usages
    @XmlElement(name = "videojuego")
    private final List<Videojuego> videojuegos;

    1 usage
    public Catalogo() { videojuegos = new ArrayList<>(); }

    3 usages
    public List<Videojuego> getVideojuegos() { return videojuegos; }

    2 usages
    public void insertarVideojuego(Videojuego videojuego){
        videojuegos.add(videojuego);
    }
}
```

Archivos

- Todos los archivos cargados son almacenados aquí.

```
package es.promehub.backend;

import ...

8 usages
public class Archivos {
    5 usages
    public static List<File> archivos = new ArrayList<>();
}
```

DatosApp

- Contiene una instancia de la clase Catalogo a la cual todos los controladores tienen acceso.

```
package es.promehub.backend;

6 usages
public class DatosApp {

    3 usages
    public static final Catalogo catalogo = new Catalogo();

}
```

Main

- Inicializa el conversor csv.

```
package es.promehub;

import ...

public class Main extends Application {
    no usages
    public static void main(String[] args) { Application.launch(Main.class, args); }

    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader fxmlLoader = new FXMLLoader(Main.class.getResource("csv_convertor.fxml"));
        Scene scene = new Scene(fxmlLoader.load(), width: 600, height: 400);
        stage.setResizable(false);
        stage.setTitle("File convertor");
        stage.setScene(scene);
        stage.show();

        Image icon = new Image(
            getClass().getResourceAsStream("es/promehub/img/convertor.png")
        );

        stage.getIcons().add(icon);
    }
}
```

- **CONTROLADORES**

CatalogConversorController

- Muestra los datos de los videojuegos almacenados en la clase catálogo.
- Contiene métodos que permiten cambiar de escena (JavaFX).

```
public class CatalogConversorController {  
  
    1 usage  
    private static final Catalogo catalogo = DatosApp.catalogo;  
  
    @FXML private VBox contenido;  
    @FXML private TextField buscador;  
    @FXML private TableView<Videojuego> tabla;  
  
    @FXML private TableColumn<Videojuego, Integer> colId;  
    @FXML private TableColumn<Videojuego, String> colTitulo;  
    @FXML private TableColumn<Videojuego, String> colPlataforma;  
    @FXML private TableColumn<Videojuego, String> colGenero;  
    @FXML private TableColumn<Videojuego, Double> colPrecio;  
    @FXML private TableColumn<Videojuego, Integer> colStock;  
    @FXML private TableColumn<Videojuego, String> colCodProveedor;  
    // fxml transition functions  
    1 usage  
    @FXML  
    public void mostrarVistaCSV() throws IOException {
```

```
        1 usage  
        private void mostrarCatalogo(){  
  
            // hacemos un observable arraylist para trabajar con javaFX  
            ObservableList<Videojuego> videojuegos = FXCollections.observableArrayList(catalogo.getVideojuegos());  
            FilteredList<Videojuego> filtrados = new FilteredList<>(videojuegos);  
            buscador.textProperty().addListener((ObservableValue<extends String> observable, String oldValue, String newValue)  
            {  
                String texto = newValue.trim().toLowerCase();  
                filtrados.setPredicate( Videojuego videojuego -> {  
                    if(texto.isEmpty()){  
                        return true;  
                    }  
                    else{  
                        return String.valueOf(videojuego.getId()).equals(texto) ||  
                               videojuego.getTitulo().toLowerCase().contains(texto);  
                    }  
                });  
            });  
  
            // le pasamos el array con el que queremos trabajar a TableView
```

CsvConversorController

- Contiene los métodos de carga y serialización CSV.
- Contiene métodos que permiten cambiar de escena (JavaFX).

```
public class CsvConversorController {  
  
    2 usages  
    private static final Catalogo catalogo = DatosApp.catalogo;  
  
    @FXML private VBox contenido;  
    // fxml transition functions  
    1 usage  
    @FXML  
    public void mostrarVistaXML() throws IOException {  
  
        FXMLLoader loader = new FXMLLoader(  
            getClass().getResource( name: "/es/promehub/xml_conversor.fxml")  
        );  
  
        Parent vista = loader.load();  
  
        contenido.getChildren().setAll(vista);  
    }  
    1 usage  
    @FXML  
    public void mostrarVistaCatalogo() throws IOException {
```

XmlConversorController

- Contiene los métodos de carga y serialización CSV.
- Contiene métodos que permiten cambiar de escena (JavaFX).

```
public class XmlConversorController {  
  
    2 usages  
    private static final Catalogo catalogo = DatosApp.catalogo;  
  
    @FXML private VBox contenido;  
    // fxml transition functions  
    1 usage  
    @FXML  
    public void mostrarVistaCSV() throws IOException {  
  
        FXMLLoader loader = new FXMLLoader(  
            | getClass().getResource( name: "/es/promehub/csv_conversor.fxml" )  
        );  
  
        Parent vista = loader.load();  
  
        contenido.getChildren().setAll(vista);  
    }  
}
```

InfoConversorController

- Crea y almacena FileCards a partir de los archivos cargados. Hace uso de la clase Archivos y su atributo tipo lista archivo.
- Contiene métodos que permiten cambiar de escena (JavaFX).

```
package es.promehub.controllers;

import ...

1 usage
public class InfoConversorController {

    @FXML private VBox contenido;
    @FXML private VBox contenedorScroll;

    1 usage
    @FXML
    public void mostrarVistaCSV() throws IOException {

        FXMLLoader loader = new FXMLLoader(
            getClass().getResource( name: "/es/promehub/csv_conversor.fxml")
        );

        Parent vista = loader.load();

        contenido.getChildren().setAll(vista);
    }
}
```

FileCardController

- Instanciado por InfoConversorController.
- Contiene información sobre un archivo.

```
package es.promehub.controllers;

import ...

2 usages
public class FileCardController {

    @FXML Label nomArchivo;
    @FXML Label tamanyo;
    @FXML Label ruta;
    @FXML Label estado;

    1 usage
    public void setDatos(String nomArchivo, long tamanyo, String ruta, boolean estado){

        this.nomArchivo.setText(nomArchivo);
        this.tamanyo.setText("Size: " + tamanyo + " bytes");
        this.ruta.setText("Path: " + ruta);
        this.estado.setText("Exists: " + estado);
    }
}
```


ANOTACIONES JAXB

Para convertir de Java a XML java necesita saber cómo convertirlo. Para ello importamos ***jakarta.xml.bind.annotation.****. Utilizamos catálogo como la etiqueta raíz del XML

```
10
11 @XmlRootElement(name = "catalogo")
12 @XmlAccessorType(XmlAccessType.FIELD)
13 public class Catalogo {
14
15     @XmlElement(name = "videojuego")
16     private final List<Videojuego> videojuegos;
17
```

```
81
82
83     try{
84         JAXBContext contexto = JAXBContext.newInstance(...classesToBeBound: Catalogo.class);
85         Marshaller marshaller = contexto.createMarshaller();
86
87         marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, value: true);
88         marshaller.marshal(catalogo, archivo);
89
90         Archivos.archivos.add(archivo);
91         System.out.println(x: "Catalog successfully exported.");
92     }catch(JAXBException e){
93         System.out.println(x: "The catalog could not be exported.");
94     }
95
96 }
97
98 @FXML
99 // 4. Cargar catálogo desde XML
100 private void cargarXML(){
101
102     // Abrimos FileChooser para que directamente seleccione el xml
103     FileChooser fileChooser = new FileChooser();
104     fileChooser.setTitle("Select XML file");
105     fileChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter("Archivos XML", "*.xml"));
106     File archivo = fileChooser.showOpenDialog(contenido.getScene().getWindow());
107
108     if (archivo != null) {
109
110         if(archivo.exists()){
111
112             Archivos.archivos.add(archivo);
113
114             try{
115                 JAXBContext contexto = JAXBContext.newInstance(...classesToBeBound: Catalogo.class);
116                 Unmarshaller unmarshaller = contexto.createUnmarshaller();
117                 Catalogo catalogoImportado = (Catalogo) unmarshaller.unmarshal(archivo);
118
119                 // insertamos cada videojuego en el catálogo original
120                 for (Videojuego videojuego : catalogoImportado.getVideojuegos()) {
121                     catalogo.insertarVideojuego(videojuego);
122                 }
123
124                 System.out.println(x: "The XML was successfully loaded");
125
126             }catch(JAXBException e){
127                 System.out.println(x: "The XML was unable to load");
128             }
129
130         }
131         else{
132             System.out.println(x: "The file does not exist");
133         }
134     }
135     else{
136         System.out.println(x: "The file cannot be null");
137     }
138
139 }
140 }
```

EXCEPCIONES

El programa contempla varios casos de error que son tratados de manera independiente informando del problema concreto. Al estar utilizando JavaFX hemos decidido que la mejor opción era imprimir estas excepciones directamente por consola.

Aquí se presentan varios ejemplos:

```
        }catch(NumberFormatException e){  
            System.out.println("This CSV does not have the correct format");  
        }  
    }  
  
    // 7. Informar de los registros procesados.  
    System.out.println("CSV successfully loaded. " + contador + " new insertions.");  
  
    }catch(IOException e){  
        System.out.println("The file was unable to open");  
    }  
}  
else{  
    System.out.println("The file does not exist");  
}  
}  
else{  
    System.out.println("The file cannot be null");  
}  
}
```